

In this chapter we delve into four deeper aspects of computability theory: (1) the recursion theorem, (2) logical theories, (3) Turing reducibility, and (4) descriptive complexity. The topic covered in each section is mainly independent of the others, except for an application of the recursion theorem at the end of the section on logical theories. Part Three of this book doesn't depend on any material from this chapter.

THE RECURSION THEOREM

To introduce the recursion theorem, we consider a paradox that arises in the study of life. It concerns the possibility of making machines that can construct replicas of themselves. The paradox can be summarized in the following manner.

1. Living things are machines.
2. Living things can self-reproduce.
3. Machines cannot self-reproduce.

Statement 1 is a tenet of modern biology. We believe that organisms operate in a mechanistic way. Statement 2 is obvious. The ability to self-reproduce is an essential characteristic of every biological species. For statement 3, we make the following argument that machines cannot self-reproduce. Consider a machine that constructs other machines, such as an automated factory that produces cars. Raw materials go in at one end, the manufacturing robots follow a set of instructions, and then completed vehicles come out the other end.

We claim that the factory must be more complex than the cars produced, in the sense that designing the factory would be more difficult than designing a car. This claim must be true because the factory itself has the car's design within it, in addition to the design of all the manufacturing robots. The same reasoning applies to any machine A that constructs a machine B : A must be *more* complex than B . But a machine cannot be more complex than itself. Consequently, no machine can construct itself, and thus self-reproduction is impossible.

How can we resolve this paradox? The answer is simple: Statement 3 is incorrect. Making machines that reproduce themselves *is* possible. The recursion theorem demonstrates how.

SELF-REFERENCE

Let's begin by making a Turing machine that ignores its input and prints out a copy of its own description. We call this machine *SELF*. To help describe *SELF*, we need the following lemma.

LEMMA 6.1

There is a computable function $q: \Sigma^* \rightarrow \Sigma^*$, where if w is any string, $q(w)$ is the description of a Turing machine P_w that prints out w and then halts.

PROOF Once we understand the statement of this lemma, the proof is easy. Obviously, we can take any string w and construct from it a Turing machine that has w built into a table so that the machine can simply output w when started. The following TM Q computes $q(w)$.

$Q =$ "On input string w :

1. Construct the following Turing machine P_w .

$P_w =$ "On any input:

1. Erase input.
2. Write w on the tape.
3. Halt."

2. Output $\langle P_w \rangle$."

.....

The Turing machine *SELF* is in two parts: *A* and *B*. We think of *A* and *B* as being two separate procedures that go together to make up *SELF*. We want *SELF* to print out $\langle SELF \rangle = \langle AB \rangle$.

Part *A* runs first and upon completion passes control to *B*. The job of *A* is to print out a description of *B*, and conversely the job of *B* is to print out a description of *A*. The result is the desired description of *SELF*. The jobs are similar, but they are carried out differently. We show how to get part *A* first.

For *A* we use the machine $P_{\langle B \rangle}$, described by $q(\langle B \rangle)$, which is the result of applying the function q to $\langle B \rangle$. Thus, part *A* is a Turing machine that prints out $\langle B \rangle$. Our description of *A* depends on having a description of *B*. So we can't complete the description of *A* until we construct *B*.

Now for part *B*. We might be tempted to define *B* with $q(\langle A \rangle)$, but that doesn't make sense! Doing so would define *B* in terms of *A*, which in turn is defined in terms of *B*. That would be a *circular* definition of an object in terms of itself, a logical transgression. Instead, we define *B* so that it prints *A* by using a different strategy: *B* computes *A* from the output that *A* produces.

We defined $\langle A \rangle$ to be $q(\langle B \rangle)$. Now comes the tricky part: If *B* can obtain $\langle B \rangle$, it can apply q to that and obtain $\langle A \rangle$. But how does *B* obtain $\langle B \rangle$? It was left on the tape when *A* finished! So *B* only needs to look at the tape to obtain $\langle B \rangle$. Then after *B* computes $q(\langle B \rangle) = \langle A \rangle$, it combines *A* and *B* into a single machine and writes its description $\langle AB \rangle = \langle SELF \rangle$ on the tape. In summary, we have:

$A = P_{\langle B \rangle}$, and

$B =$ "On input $\langle M \rangle$, where M is a portion of a TM:

1. Compute $q(\langle M \rangle)$.
2. Combine the result with $\langle M \rangle$ to make a complete TM.
3. Print the description of this TM and halt."

This completes the construction of *SELF*, for which a schematic diagram is presented in the following figure.

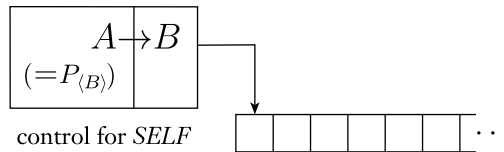


FIGURE 6.2
Schematic of *SELF*, a TM that prints its own description

If we now run *SELF*, we observe the following behavior.

1. First *A* runs. It prints $\langle B \rangle$ on the tape.
2. *B* starts. It looks at the tape and finds its input, $\langle B \rangle$.
3. *B* calculates $q(\langle B \rangle) = \langle A \rangle$ and combines that with $\langle B \rangle$ into a TM description, $\langle SELF \rangle$.
4. *B* prints this description and halts.

We can easily implement this construction in any programming language to obtain a program that outputs a copy of itself. We can even do so in plain English. Suppose that we want to give an English sentence that commands the reader to print a copy of the same sentence. One way to do so is to say:

Print out this sentence.

This sentence has the desired meaning because it directs the reader to print a copy of the sentence itself. However, it doesn't have an obvious translation into a programming language because the self-referential word "this" in the sentence usually has no counterpart. But no self-reference is needed to make such a sentence. Consider the following alternative.

Print out two copies of the following, the second one in quotes:
 "Print out two copies of the following, the second one in quotes:"

In this sentence, the self-reference is replaced with the same construction used to make the TM *SELF*. Part *B* of the construction is the clause:

Print out two copies of the following, the second one in quotes:

Part *A* is the same, with quotes around it. *A* provides a copy of *B* to *B* so *B* can process that copy as the TM does.

The recursion theorem provides the ability to implement the self-referential *this* into any programming language. With it, any program has the ability to refer to its own description, which has certain applications, as you will see. Before getting to that, we state the recursion theorem itself. The recursion theorem extends the technique we used in constructing *SELF* so that a program can obtain its own description and then go on to compute with it, instead of merely printing it out.

THEOREM 6.3

Recursion theorem Let T be a Turing machine that computes a function $t: \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$. There is a Turing machine R that computes a function $r: \Sigma^* \rightarrow \Sigma^*$, where for every w ,

$$r(w) = t(\langle R \rangle, w).$$

The statement of this theorem seems a bit technical, but it actually represents something quite simple. To make a Turing machine that can obtain its own description and then compute with it, we need only make a machine, called T

in the statement, that receives the description of the machine as an extra input. Then the recursion theorem produces a new machine R , which operates exactly as T does but with R 's description filled in automatically.

PROOF The proof is similar to the construction of *SELF*. We construct a TM R in three parts, A , B , and T , where T is given by the statement of the theorem; a schematic diagram is presented in the following figure.

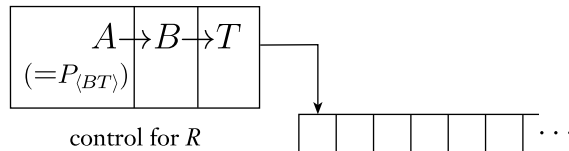


FIGURE 6.4
Schematic of R

Here, A is the Turing machine $P_{\langle BT \rangle}$ described by $q(\langle BT \rangle)$. To preserve the input w , we redesign q so that $P_{\langle BT \rangle}$ writes its output following any string preexisting on the tape. After A runs, the tape contains $w\langle BT \rangle$.

Again, B is a procedure that examines its tape and applies q to its contents. The result is $\langle A \rangle$. Then B combines A , B , and T into a single machine and obtains its description $\langle ABT \rangle = \langle R \rangle$. Finally, it encodes that description together with w , places the resulting string $\langle R, w \rangle$ on the tape, and passes control to T .

TERMINOLOGY FOR THE RECURSION THEOREM

The recursion theorem states that Turing machines can obtain their own description and then go on to compute with it. At first glance, this capability may seem to be useful only for frivolous tasks such as making a machine that prints a copy of itself. But, as we demonstrate, the recursion theorem is a handy tool for solving certain problems concerning the theory of algorithms.

You can use the recursion theorem in the following way when designing Turing machine algorithms. If you are designing a machine M , you can include the phrase “obtain own description $\langle M \rangle$ ” in the informal description of M 's algorithm. Upon having obtained its own description, M can then go on to use it as it would use any other computed value. For example, M might simply print out $\langle M \rangle$ as happens in the TM *SELF*, or it might count the number of states in $\langle M \rangle$, or possibly even simulate $\langle M \rangle$. To illustrate this method, we use the recursion theorem to describe the machine *SELF*.

$SELF =$ “On any input:

1. Obtain, via the recursion theorem, own description $\langle SELF \rangle$.
2. Print $\langle SELF \rangle$.”

The recursion theorem shows how to implement the “obtain own description” construct. To produce the machine $SELF$, we first write the following machine T .

$T =$ “On input $\langle M, w \rangle$:

1. Print $\langle M \rangle$ and halt.”

The TM T receives a description of a TM M and a string w as input, and it prints the description of M . Then the recursion theorem shows how to obtain a TM R , which on input w operates like T on input $\langle R, w \rangle$. Thus, R prints the description of R —exactly what is required of the machine $SELF$.

APPLICATIONS

A **computer virus** is a computer program that is designed to spread itself among computers. Aptly named, it has much in common with a biological virus. Computer viruses are inactive when standing alone as a piece of code. But when placed appropriately in a host computer, thereby “infecting” it, they can become activated and transmit copies of themselves to other accessible machines. Various media can transmit viruses, including the Internet and transferable disks. In order to carry out its primary task of self-replication, a virus may contain the construction described in the proof of the recursion theorem.

Let’s now consider three theorems whose proofs use the recursion theorem. An additional application appears in the proof of Theorem 6.17 in Section 6.2.

First we return to the proof of the undecidability of A_{TM} . Recall that we earlier proved it in Theorem 4.11, using Cantor’s diagonal method. The recursion theorem gives us a new and simpler proof.

THEOREM 6.5

A_{TM} is undecidable.

PROOF We assume that Turing machine H decides A_{TM} , for the purpose of obtaining a contradiction. We construct the following machine B .

$B =$ “On input w :

1. Obtain, via the recursion theorem, own description $\langle B \rangle$.
2. Run H on input $\langle B, w \rangle$.
3. Do the opposite of what H says. That is, *accept* if H rejects and *reject* if H accepts.”

Running B on input w does the opposite of what H declares it does. Therefore, H cannot be deciding A_{TM} . Done!

The following theorem concerning minimal Turing machines is another application of the recursion theorem.

DEFINITION 6.6

If M is a Turing machine, then we say that the *length* of the description $\langle M \rangle$ of M is the number of symbols in the string describing M . Say that M is **minimal** if there is no Turing machine equivalent to M that has a shorter description. Let

$$MIN_{TM} = \{\langle M \rangle \mid M \text{ is a minimal TM}\}.$$

THEOREM 6.7

MIN_{TM} is not Turing-recognizable.

PROOF Assume that some TM E enumerates MIN_{TM} and obtain a contradiction. We construct the following TM C .

$C =$ “On input w :

1. Obtain, via the recursion theorem, own description $\langle C \rangle$.
2. Run the enumerator E until a machine D appears with a longer description than that of C .
3. Simulate D on input w .”

Because MIN_{TM} is infinite, E ’s list must contain a TM with a longer description than C ’s description. Therefore, step 2 of C eventually terminates with some TM D that is longer than C . Then C simulates D and so is equivalent to it. Because C is shorter than D and is equivalent to it, D cannot be minimal. But D appears on the list that E produces. Thus, we have a contradiction.

Our final application of the recursion theorem is a type of fixed-point theorem. A **fixed point** of a function is a value that isn’t changed by application of the function. In this case, we consider functions that are computable transformations of Turing machine descriptions. We show that for any such transformation, some Turing machine exists whose behavior is unchanged by the transformation. This theorem is called the fixed-point version of the recursion theorem.

THEOREM 6.8

Let $t: \Sigma^* \rightarrow \Sigma^*$ be a computable function. Then there is a Turing machine F for which $t(\langle F \rangle)$ describes a Turing machine equivalent to F . Here we'll assume that if a string isn't a proper Turing machine encoding, it describes a Turing machine that always rejects immediately.

In this theorem, t plays the role of the transformation, and F is the fixed point.

PROOF Let F be the following Turing machine.

F = "On input w :

1. Obtain, via the recursion theorem, own description $\langle F \rangle$.
2. Compute $t(\langle F \rangle)$ to obtain the description of a TM G .
3. Simulate G on w ."

Clearly, $\langle F \rangle$ and $t(\langle F \rangle) = \langle G \rangle$ describe equivalent Turing machines because F simulates G .

6.2

DECIDABILITY OF LOGICAL THEORIES

Mathematical logic is the branch of mathematics that investigates mathematics itself. It addresses questions such as: What is a theorem? What is a proof? What is truth? Can an algorithm decide which statements are true? Are all true statements provable? We'll touch on a few of these topics in our brief introduction to this rich and fascinating subject.

We focus on the problem of determining whether mathematical statements are true or false and investigate the decidability of this problem. The answer depends on the domain of mathematics from which the statements are drawn. We examine two domains: one for which we can give an algorithm to decide truth, and another for which this problem is undecidable.

First, we need to set up a precise language to formulate these problems. Our intention is to be able to consider mathematical statements such as

1. $\forall q \exists p \forall x, y [p > q \wedge (x, y > 1 \rightarrow xy \neq p)]$,
2. $\forall a, b, c, n [(a, b, c > 0 \wedge n > 2) \rightarrow a^n + b^n \neq c^n]$, and
3. $\forall q \exists p \forall x, y [p > q \wedge (x, y > 1 \rightarrow (xy \neq p \wedge xy \neq p + 2))]$.

Statement 1 says that infinitely many prime numbers exist, which has been known to be true since the time of Euclid, about 2,300 years ago. Statement 2 is *Fermat's last theorem*, which has been known to be true only since Andrew Wiles proved it in 1994. Finally, statement 3 says that infinitely many prime pairs¹ exist. Known as the *twin prime conjecture*, it remains unsolved.

¹*Prime pairs* are primes that differ by 2.

To consider whether we could automate the process of determining which of these statements are true, we treat such statements merely as strings and define a language consisting of those statements that are true. Then we ask whether this language is decidable.

To make this a bit more precise, let's describe the form of the alphabet of this language:

$$\{\wedge, \vee, \neg, (,), \forall, \exists, x, R_1, \dots, R_k\}.$$

The symbols $\wedge$, $\vee$, and $\neg$ are called **Boolean operations**; “(” and “)” are the **parentheses**; the symbols $\forall$ and $\exists$ are called **quantifiers**; the symbol x is used to denote **variables**;² and the symbols $R_1, \dots, R_k$ are called **relations**.

A **formula** is a well-formed string over this alphabet. For completeness, we'll sketch the technical but obvious definition of a **well-formed formula** here, but feel free to skip this part and go on to the next paragraph. A string of the form $R_i(x_1, \dots, x_k)$ is an **atomic formula**. The value j is the **arity** of the relation symbol R_i . All appearances of the same relation symbol in a well-formed formula must have the same arity. Subject to this requirement, a string ϕ is a formula if it

1. is an atomic formula,
2. has the form $\phi_1 \wedge \phi_2$ or $\phi_1 \vee \phi_2$ or $\neg\phi_1$, where ϕ_1 and ϕ_2 are smaller formulas, or
3. has the form $\exists x_i [\phi_1]$ or $\forall x_i [\phi_1]$, where ϕ_1 is a smaller formula.

A quantifier may appear anywhere in a mathematical statement. Its **scope** is the fragment of the statement appearing within the matched pair of parentheses or brackets following the quantified variable. We assume that all formulas are in **prenex normal form**, where all quantifiers appear in the front of the formula. A variable that isn't bound within the scope of a quantifier is called a **free variable**. A formula with no free variables is called a **sentence** or **statement**.

EXAMPLE 6.9

Among the following examples of formulas, only the last one is a sentence.

1. $R_1(x_1) \wedge R_2(x_1, x_2, x_3)$
2. $\forall x_1 [R_1(x_1) \wedge R_2(x_1, x_2, x_3)]$
3. $\forall x_1 \exists x_2 \exists x_3 [R_1(x_1) \wedge R_2(x_1, x_2, x_3)]$ ■

Having established the syntax of formulas, let's discuss their meanings. The Boolean operations and the quantifiers have their usual meanings. But to determine the meaning of the variables and relation symbols, we need to specify two items. One is the **universe** over which the variables may take values. The other

²If we need to write several variables in a formula, we use the symbols w, y, z , or x_1, x_2, x_3 , and so on. We don't list all the infinitely many possible variables in the alphabet to keep the alphabet finite. Instead, we list only the variable symbol x , and use strings of x 's to indicate other variables, as in xx for x_2 , xxx for x_3 , and so on.

is an assignment of specific relations to the relation symbols. As we described in Section 0.2 (page 9), a relation is a function from k -tuples over the universe to $\{\text{TRUE}, \text{FALSE}\}$. The arity of a relation symbol must match that of its assigned relation.

A universe together with an assignment of relations to relation symbols is called a **model**.³ Formally, we say that a model $\mathcal{M}$ is a tuple $(U, P_1, \dots, P_k)$, where U is the universe and P_1 through P_k are the relations assigned to symbols R_1 through R_k . We sometimes refer to the **language of a model** to be the collection of formulas that use only the relation symbols the model assigns, and that use each relation symbol with the correct arity. If ϕ is a sentence in the language of a model, ϕ is either true or false in that model. If ϕ is true in a model $\mathcal{M}$, we say that $\mathcal{M}$ is a model of ϕ .

If you feel overwhelmed by these definitions, concentrate on our objective in stating them. We want to set up a precise language of mathematical statements so that we can ask whether an algorithm can determine which are true and which are false. The following two examples should be helpful.

EXAMPLE 6.10

Let ϕ be the sentence $\forall x \forall y [R_1(x, y) \vee R_1(y, x)]$. Let model $\mathcal{M}_1 = (\mathcal{N}, \leq)$ be the model whose universe is the natural numbers and that assigns the “less than or equal” relation to the symbol R_1 . Obviously, ϕ is true in model $\mathcal{M}_1$ because either $a \leq b$ or $b \leq a$ for any two natural numbers a and b . However, if $\mathcal{M}_1$ assigned “less than” instead of “less than or equal” to R_1 , then ϕ would not be true because it fails when x and y are equal.

If we know in advance which relation will be assigned to R_i , we may use the customary symbol for that relation in place of R_i with infix notation rather than prefix notation if customary for that symbol. Thus, with model $\mathcal{M}_1$ in mind, we could write ϕ as $\forall x \forall y [x \leq y \vee y \leq x]$. ■

EXAMPLE 6.11

Now let $\mathcal{M}_2$ be the model whose universe is the real numbers $\mathcal{R}$ and that assigns the relation *PLUS* to R_1 , where $PLUS(a, b, c) = \text{TRUE}$ whenever $a + b = c$. Then $\mathcal{M}_2$ is a model of $\psi = \forall y \exists x [R_1(x, x, y)]$. However, if $\mathcal{N}$ were used for the universe instead of $\mathcal{R}$ in $\mathcal{M}_2$, the sentence would be false.

As in Example 6.10, we may write ψ as $\forall y \exists x [x + x = y]$ in place of $\forall y \exists x [R_1(x, x, y)]$ when we know in advance that we will be assigning the addition relation to R_1 . ■

As Example 6.11 illustrates, we can represent functions such as the addition function by relations. Similarly, we can represent constants such as 0 and 1 by relations.

³A model is also variously called an *interpretation* or a *structure*.

Now we give one final definition in preparation for the next section. If $\mathcal{M}$ is a model, we let the *theory of $\mathcal{M}$* , written $\text{Th}(\mathcal{M})$, be the collection of true sentences in the language of that model.

A DECIDABLE THEORY

Number theory is one of the oldest branches of mathematics and also one of its most difficult. Many innocent-looking statements about the natural numbers with the plus and times operations have confounded mathematicians for centuries, such as the twin prime conjecture mentioned earlier.

In one of the celebrated developments in mathematical logic, Alonzo Church, building on the work of Kurt Gödel, showed that no algorithm can decide in general whether statements in number theory are true or false. Formally, we write $(\mathcal{N}, +, \times)$ to be the model whose universe is the natural numbers⁴ with the usual $+$ and $\times$ relations. Church showed that $\text{Th}(\mathcal{N}, +, \times)$, the theory of this model, is undecidable.

Before looking at this undecidable theory, let's examine one that is decidable. Let $(\mathcal{N}, +)$ be the same model, without the $\times$ relation. Its theory is $\text{Th}(\mathcal{N}, +)$. For example, the formula $\forall x \exists y [x + x = y]$ is true and is therefore a member of $\text{Th}(\mathcal{N}, +)$, but the formula $\exists y \forall x [x + x = y]$ is false and is therefore not a member.

THEOREM 6.12

$\text{Th}(\mathcal{N}, +)$ is decidable.

PROOF IDEA This proof is an interesting and nontrivial application of the theory of finite automata that we presented in Chapter 1. One fact about finite automata that we use appears in Problem 1.32, (page 88) where you were asked to show that they are capable of doing addition if the input is presented in a special form. The input describes three numbers in parallel by representing one bit of each number in a single symbol from an eight-symbol alphabet. Here we use a generalization of this method to present i -tuples of numbers in parallel using an alphabet with 2^i symbols.

We give an algorithm that can determine whether its input, a sentence ϕ in the language of $(\mathcal{N}, +)$, is true in that model. Let

$$\phi = Q_1 x_1 Q_2 x_2 \cdots Q_l x_l [\psi],$$

where $Q_1, \dots, Q_l$ each represents either $\exists$ or $\forall$ and ψ is a formula without quantifiers that has variables $x_1, \dots, x_l$. For each i from 0 to l , define formula ϕ_i as

$$\phi_i = Q_{i+1} x_{i+1} Q_{i+2} x_{i+2} \cdots Q_l x_l [\psi].$$

Thus $\phi_0 = \phi$ and $\phi_l = \psi$.

⁴For convenience in this chapter, we change our usual definition of $\mathcal{N}$ to be $\{0, 1, 2, \dots\}$.

Formula ϕ_i has i free variables. For $a_1, \dots, a_i \in \mathcal{N}$, write $\phi_i(a_1, \dots, a_i)$ to be the sentence obtained by substituting the constants $a_1, \dots, a_i$ for the variables $x_1, \dots, x_i$ in ϕ_i .

For each i from 0 to l , the algorithm constructs a finite automaton A_i that recognizes the collection of strings representing i -tuples of numbers that make ϕ_i true. The algorithm begins by constructing A_l directly, using a generalization of the method in the solution to Problem 1.32. Then, for each i from l down to 1, it uses A_i to construct A_{i-1} . Finally, once the algorithm has A_0 , it tests whether A_0 accepts the empty string. If it does, ϕ is true and the algorithm accepts.

PROOF For $i > 0$, define the alphabet

$$\Sigma_i = \left\{ \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ \vdots \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ \vdots \\ 1 \\ 1 \end{bmatrix}, \dots, \begin{bmatrix} 1 \\ \vdots \\ 1 \\ 1 \end{bmatrix} \right\}.$$

Hence Σ_i contains all size i columns of 0s and 1s. A string over Σ_i represents i binary integers (reading across the rows). We also define $\Sigma_0 = \{[]\}$, where $[]$ is a symbol.

We now present an algorithm that decides $\text{Th}(\mathcal{N}, +)$. On input ϕ , where ϕ is a sentence, the algorithm operates as follows. Write ϕ and define ϕ_i for each i from 0 to l , as in the proof idea. For each such i , construct a finite automaton A_i from ϕ_i that accepts strings over Σ_i corresponding to i -tuples $a_1, \dots, a_i$ whenever $\phi_i(a_1, \dots, a_i)$ is true, as follows.

To construct the first machine A_l , observe that $\phi_l = \psi$ is a Boolean combination of atomic formulas. An atomic formula in the language of $\text{Th}(\mathcal{N}, +)$ is a single addition. Finite automata can be constructed to compute any of these individual relations corresponding to a single addition and then combined to give the automaton A_l . Doing so involves the use of the regular language closure constructions for union, intersection, and complementation to compute Boolean combinations of the atomic formulas.

Next, we show how to construct A_i from A_{i+1} . If $\phi_i = \exists x_{i+1} \phi_{i+1}$, we construct A_i to operate as A_{i+1} operates, except that it nondeterministically guesses the value of a_{i+1} instead of receiving it as part of the input.

More precisely, A_i contains a state for each A_{i+1} state and a new start state. Every time A_i reads a symbol

$$\begin{bmatrix} b_1 \\ \vdots \\ b_{i-1} \\ b_i \end{bmatrix},$$

where every $b_j \in \{0, 1\}$ is a bit of the number a_j , it nondeterministically guesses $z \in \{0, 1\}$ and simulates A_{i+1} on the input symbol

$$\begin{bmatrix} b_1 \\ \vdots \\ b_{i-1} \\ b_i \\ z \end{bmatrix}.$$

Initially, A_i nondeterministically guesses the leading bits of a_{i+1} corresponding to suppressed leading 0s in a_1 through a_i by nondeterministically branching using ϵ -transitions from its new start state to all states that A_{i+1} could reach from its start state with input strings of the symbols

$$\left\{ \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix} \right\}$$

in Σ_{i+1} . Clearly, A_i accepts its input $(a_1, \dots, a_i)$ if some a_{i+1} exists where A_{i+1} accepts $(a_1, \dots, a_{i+1})$.

If $\phi_i = \forall x_{i+1} \phi_{i+1}$, it is equivalent to $\neg \exists x_{i+1} \neg \phi_{i+1}$. Thus, we can construct the finite automaton that recognizes the complement of the language of A_{i+1} , then apply the preceding construction for the $\exists$ quantifier, and finally apply complementation once again to obtain A_i .

Finite automaton A_0 accepts any input iff ϕ_0 is true. So the final step of the algorithm tests whether A_0 accepts ϵ . If it does, ϕ is true and the algorithm accepts; otherwise, it rejects.

AN UNDECIDABLE THEORY

As we mentioned earlier, $\text{Th}(\mathcal{N}, +, \times)$ is an undecidable theory. No algorithm exists for deciding the truth or falsity of mathematical statements, even when restricted to the language of $(\mathcal{N}, +, \times)$. This theorem has great importance philosophically because it demonstrates that mathematics cannot be mechanized. We state this theorem, but give only a brief sketch of its proof.

THEOREM 6.13

$\text{Th}(\mathcal{N}, +, \times)$ is undecidable.

Although it contains many details, the proof of this theorem is not difficult conceptually. It follows the pattern of the other proofs of undecidability presented in Chapter 4. We show that $\text{Th}(\mathcal{N}, +, \times)$ is undecidable by reducing A_{TM} to it, using the computation history method as previously described (page 220). The existence of the reduction depends on the following lemma.

LEMMA 6.14

Let M be a Turing machine and w a string. We can construct from M and w a formula $\phi_{M,w}$ in the language of $(\mathcal{N}, +, \times)$ that contains a single free variable x , whereby the sentence $\exists x \phi_{M,w}$ is true iff M accepts w .

PROOF IDEA Formula $\phi_{M,w}$ “says” that x is a (suitably encoded) accepting computation history of M on w . Of course, x actually is just a rather large integer, but it represents a computation history in a form that can be checked by using the $+$ and $\times$ operations.

The actual construction of $\phi_{M,w}$ is too complicated to present here. It extracts individual symbols in the computation history with the $+$ and $\times$ operations to check that the start configuration for M on w is correct, that each configuration legally follows from the one preceding it, and that the last configuration is accepting.

PROOF OF THEOREM 6.13 We give a mapping reduction from A_{TM} to $\text{Th}(\mathcal{N}, +, \times)$. The reduction constructs the formula $\phi_{M,w}$ from the input $\langle M, w \rangle$ by using Lemma 6.14. Then it outputs the sentence $\exists x \phi_{M,w}$.

Next, we sketch the proof of Kurt Gödel's celebrated *incompleteness theorem*. Informally, this theorem says that in any reasonable system of formalizing the notion of provability in number theory, some true statements are unprovable.

Loosely speaking, the **formal proof** π of a statement ϕ is a sequence of statements, $S_1, S_2, \dots, S_l$, where $S_l = \phi$. Each S_i follows from the preceding statements and certain basic axioms about numbers, using simple and precise rules of implication. We don't have space to define the concept of proof, but for our purposes, assuming the following two reasonable properties of proofs will be enough.

1. The correctness of a proof of a statement can be checked by machine. Formally, $\{\langle \phi, \pi \rangle \mid \pi \text{ is a proof of } \phi\}$ is decidable.
2. The system of proofs is *sound*. That is, if a statement is provable (i.e., has a proof), it is true.

If a system of provability satisfies these two conditions, the following three theorems hold.

THEOREM 6.15

The collection of provable statements in $\text{Th}(\mathcal{N}, +, \times)$ is Turing-recognizable.

PROOF The following algorithm P accepts its input ϕ if ϕ is provable. Algorithm P tests each string as a candidate for a proof π of ϕ , using the proof checker assumed in provability property 1. If it finds that any of these candidates is a proof, it accepts.

Now we can use the preceding theorem to prove our version of the incompleteness theorem.

THEOREM 6.16

Some true statement in $\text{Th}(\mathcal{N}, +, \times)$ is not provable.

PROOF We give a proof by contradiction. We assume to the contrary that all true statements are provable. Using this assumption, we describe an algorithm D that decides whether statements are true, contradicting Theorem 6.13.

On input ϕ , algorithm D operates by running algorithm P given in the proof of Theorem 6.15 in parallel on inputs ϕ and $\neg\phi$. One of these two statements is true and thus by our assumption is provable. Therefore, P must halt on one of the two inputs. By provability property 2, if ϕ is provable, then ϕ is true; and if $\neg\phi$ is provable, then ϕ is false. So algorithm D can decide the truth or falsity of ϕ .

In the final theorem of this section, we use the recursion theorem to give an explicit sentence in the language of $(\mathcal{N}, +, \times)$ that is true but not provable. In Theorem 6.16 we demonstrated the existence of such a sentence but didn't actually describe one, as we do now.

THEOREM 6.17

The sentence $\psi_{\text{unprovable}}$, as described in the proof, is unprovable.

PROOF IDEA Construct a sentence that says “This sentence is not provable,” using the recursion theorem to obtain the self-reference.

PROOF Let S be a TM that operates as follows.

$S =$ “On any input:

1. Obtain own description $\langle S \rangle$ via the recursion theorem.
2. Construct the sentence $\psi = \neg\exists c [\phi_{S,0}]$, using Lemma 6.14.
3. Run algorithm P from the proof of Theorem 6.15 on input ψ .
4. If stage 3 accepts, *accept*.”

Let $\psi_{\text{unprovable}}$ be the sentence ψ described in stage 2 of algorithm S . That sentence is true iff S doesn't accept 0 (the string 0 was selected arbitrarily).

If S finds a proof of $\psi_{\text{unprovable}}$, S accepts 0, and the sentence would thus be false. A false sentence cannot be provable, so this situation cannot occur. The only remaining possibility is that S fails to find a proof of $\psi_{\text{unprovable}}$ and so S doesn't accept 0. But then $\psi_{\text{unprovable}}$ is true, as we claimed.

6.3

TURING REDUCIBILITY

We introduced the reducibility concept in Chapter 5 as a way of using a solution to one problem to solve other problems. Thus, if A is reducible to B , and we find a solution to B , we can obtain a solution to A . Subsequently, we described *mapping reducibility*, a specific form of reducibility. But does mapping reducibility capture our intuitive concept of reducibility in the most general way? It doesn't.

For example, consider the two languages A_{TM} and $\overline{A_{\text{TM}}}$. Intuitively, they are reducible to one another because a solution to either could be used to solve the other by simply reversing the answer. However, we know that $\overline{A_{\text{TM}}}$ is *not* mapping reducible to A_{TM} because A_{TM} is Turing-recognizable but $\overline{A_{\text{TM}}}$ isn't. Here we present a very general form of reducibility, called **Turing reducibility**, which captures our intuitive concept of reducibility more closely.

DEFINITION 6.18

An **oracle** for a language B is an external device that is capable of reporting whether any string w is a member of B . An **oracle Turing machine** is a modified Turing machine that has the additional capability of querying an oracle. We write M^B to describe an oracle Turing machine that has an oracle for language B .

We aren't concerned with the way the oracle determines its responses. We use the term oracle to connote a magical ability and consider oracles for languages that aren't decidable by ordinary algorithms, as the following example shows.

EXAMPLE 6.19

Consider an oracle for A_{TM} . An oracle Turing machine with an oracle for A_{TM} can decide more languages than an ordinary Turing machine can. Such a machine can (obviously) decide A_{TM} itself, by querying the oracle about the input. It can also decide E_{TM} , the emptiness testing problem for TMs with the following procedure called $T^{A_{\text{TM}}}$.

$T^{A_{\text{TM}}} =$ "On input $\langle M \rangle$, where M is a TM:

1. Construct the following TM N .
 $N =$ "On any input:
 1. Run M in parallel on all strings in Σ^* .
 2. If M accepts any of these strings, *accept*."
2. Query the oracle to determine whether $\langle N, 0 \rangle \in A_{\text{TM}}$.
3. If the oracle answers NO, *accept*; if YES, *reject*."

We start with an example. Consider the information content of the following two binary sequences.

$$A = 01$$

$$B = 1110010110100011101010000111010011010111$$

Intuitively, sequence A contains little information because it is merely a repetition of the pattern 01 twenty times. In contrast, sequence B appears to contain more information.

We can use this simple example to illustrate the idea behind the definition of information that we present. We define the quantity of information contained in an object to be the size of that object's smallest representation or description. By a description of an object, we mean a precise and unambiguous characterization of the object so that we may recreate it from the description alone. Thus, sequence A contains little information because it has a small description, whereas sequence B apparently contains more information because it seems to have no concise description.

Why do we consider only the *shortest* description when determining an object's quantity of information? We may always describe an object, such as a string, by placing a copy of the object directly into the description. Thus, we can obviously describe the preceding string B with a table that is 40 bits long containing a copy of B . This type of description is never shorter than the object itself and doesn't tell us anything about its information quantity. However, a description that is significantly shorter than the object implies that the information contained within it can be compressed into a small volume, and so the amount of information can't be very large. Hence the size of the shortest description determines the amount of information.

Now we formalize this intuitive idea. Doing so isn't difficult, but we must do some preliminary work. First, we restrict our attention to objects that are binary strings. Other objects can be represented as binary strings, so this restriction doesn't limit the scope of the theory. Second, we consider only descriptions that are themselves binary strings. By imposing this requirement, we may easily compare the length of the object with the length of its description. In the next section, we consider the type of description that we allow.

MINIMAL LENGTH DESCRIPTIONS

Many types of description language can be used to define information. Selecting which language to use affects the characteristics of the definition. Our description language is based on algorithms.

One way to use algorithms to describe strings is to construct a Turing machine that prints out the string when it is started on a blank tape and then represent that Turing machine itself as a string. Thus, the string representing the Turing machine is a description of the original string. A drawback to this approach is that a Turing machine cannot represent a table of information concisely with its transition function. To represent a string of n bits, you might use n states and n rows in the transition function table. That would result in a description that is excessively long for our purpose. Instead, we use the following more concise description language.

We describe a binary string x with a Turing machine M and a binary input w to M . The length of the description is the combined length of representing M and w . We write this description with our usual notation for encoding several objects into a single binary string $\langle M, w \rangle$. But here we must pay additional attention to the encoding operation $\langle \cdot, \cdot \rangle$ because we need to produce a concise result. We define the string $\langle M, w \rangle$ to be $\langle M \rangle w$, where we simply concatenate the binary string w onto the end of the binary encoding of M . The encoding $\langle M \rangle$ of M may be done in any standard way, except for the subtlety that we describe in the next paragraph. (Don't worry about this subtle point on your first reading of this material. For now, skip past the next paragraph and the following figure.)

When concatenating w onto the end of $\langle M \rangle$ to yield a description of x , you might run into trouble if the point at which $\langle M \rangle$ ends and w begins is not discernible from the description itself. Otherwise, several ways of partitioning the description $\langle M \rangle w$ into a syntactically correct TM and an input may occur, and then the description would be ambiguous and hence invalid. We avoid this problem by ensuring that we can locate the separation between $\langle M \rangle$ and w in $\langle M \rangle w$. One way to do so is to write each bit of $\langle M \rangle$ twice, writing 0 as 00 and 1 as 11, and then follow it with 01 to mark the separation point. We illustrate this idea in the following figure, depicting the description $\langle M, w \rangle$ of some string x .

$$\langle M, w \rangle = \underbrace{11001111001100 \cdots 1100}_{\langle M \rangle} \overbrace{01}^{\text{delimiter}} \underbrace{01101011 \cdots 010}_w$$

FIGURE 6.22

Example of the format of the description $\langle M, w \rangle$ of some string x

Now that we have fixed our description language, we are ready to define our measure of the quantity of information in a string.

DEFINITION 6.23

Let x be a binary string. The *minimal description* of x , written $d(x)$, is the shortest string $\langle M, w \rangle$ where TM M on input w halts with x on its tape. If several such strings exist, select the lexicographically first among them. The *descriptive complexity*⁵ of x , written $K(x)$, is

$$K(x) = |d(x)|.$$

In other words, $K(x)$ is the length of the minimal description of x . The definition of $K(x)$ is intended to capture our intuition for the amount of information in the string x . Next we establish some simple results about descriptive complexity.

THEOREM 6.24

$$\exists c \forall x [K(x) \leq |x| + c]$$

This theorem says that the descriptive complexity of a string is at most a fixed constant more than its length. The constant is a universal one, not dependent on the string.

PROOF To prove an upper bound on $K(x)$ as this theorem claims, we need only demonstrate some description of x that is no longer than the stated bound. Then the minimal description of x may be shorter than the demonstrated description, but not longer.

Consider the following description of the string x . Let M be a Turing machine that halts as soon as it is started. This machine computes the identity function—its output is the same as its input. A description of x is simply $\langle M \rangle x$. Letting c be the length of $\langle M \rangle$ completes the proof.

Theorem 6.24 illustrates how we use the input to the Turing machine to represent information that would require a significantly larger description if stored instead by using the machine's transition function. It conforms to our intuition that the amount of information contained by a string cannot be (substantially) more than its length. Similarly, intuition says that the information contained by the string xx is not significantly more than the information contained by x . The following theorem verifies this fact.

⁵*Descriptive complexity* is called *Kolmogorov complexity* or *Kolmogorov–Chaitin complexity* in some treatments.

THEOREM 6.25

$$\exists c \forall x [K(xx) \leq K(x) + c]$$

PROOF Consider the following Turing machine M , which expects an input of the form $\langle N, w \rangle$, where N is a Turing machine and w is an input for it.

M = “On input $\langle N, w \rangle$, where N is a TM and w is a string:

1. Run N on w until it halts and produces an output string s .
2. Output the string ss .”

A description of xx is $\langle M \rangle d(x)$. Recall that $d(x)$ is a minimal description of x . The length of this description is $|\langle M \rangle| + |d(x)|$, which is $c + K(x)$ where c is the length of $\langle M \rangle$.

Next we examine how the descriptive complexity of the concatenation xy of two strings x and y is related to their individual complexities. Theorem 6.24 might lead us to believe that the complexity of the concatenation is at most the sum of the individual complexities (plus a fixed constant), but the cost of combining two descriptions leads to a greater bound, as described in the following theorem.

THEOREM 6.26

$$\exists c \forall x, y [K(xy) \leq 2K(x) + K(y) + c]$$

PROOF We construct a TM M that breaks its input w into two separate descriptions. The bits of the first description $d(x)$ are all doubled and terminated with string 01 before the second description $d(y)$ appears, as described in the text preceding Figure 6.22. Once both descriptions have been obtained, they are run to obtain the strings x and y and the output xy is produced.

The length of this description of xy is clearly twice the complexity of x plus the complexity of y plus a fixed constant for describing M . This sum is

$$2K(x) + K(y) + c,$$

and the proof is complete.

We may improve this theorem somewhat by using a more efficient method of indicating the separation between the two descriptions. One way avoids doubling the bits of $d(x)$. Instead we prepend the length of $d(x)$ as a binary integer that has been doubled to differentiate it from $d(x)$. The description still contains enough information to decode it into the two descriptions of x and y , and it now has length at most

$$2 \log_2(K(x)) + K(x) + K(y) + c.$$

Further small improvements are possible. However, as Problem 6.26 asks you to show, we cannot reach the bound $K(x) + K(y) + c$.

OPTIMALITY OF THE DEFINITION

Now that we have established some of the elementary properties of descriptive complexity and you have had a chance to develop some intuition, we discuss some features of the definitions.

Our definition of $K(x)$ has an optimality property among all possible ways of defining descriptive complexity with algorithms. Suppose that we consider a general *description language* to be any computable function $p: \Sigma^* \rightarrow \Sigma^*$ and define the minimal description of x with respect to p , written $d_p(x)$, to be the first string s where $p(s) = x$, in the standard string order. Thus, s is lexicographically first among the shortest descriptions of x . Define $K_p(x) = |d_p(x)|$.

For example, consider a programming language such as Python (encoded into binary) as the description language. Then $d_{\text{Python}}(x)$ would be the minimal Python program that outputs x , and $K_{\text{Python}}(x)$ would be the length of the minimal program.

The following theorem shows that any description language of this type is not significantly more concise than the language of Turing machines and inputs that we originally defined.

THEOREM 6.27

For any description language p , a fixed constant c exists that depends only on p , where

$$\forall x [K(x) \leq K_p(x) + c].$$

PROOF IDEA We illustrate the idea of this proof by using the Python example. Suppose that x has a short description w in Python. Let M be a TM that can interpret Python and use the Python program for x as M 's input w . Then $\langle M, w \rangle$ is a description of x that is only a fixed amount larger than the Python description of x . The extra length is for the Python interpreter M .

PROOF Take any description language p and consider the following Turing machine M .

$M =$ “On input w :

1. Output $p(w)$.”

Then $\langle M \rangle d_p(x)$ is a description of x whose length is at most a fixed constant greater than $K_p(x)$. The constant is the length of $\langle M \rangle$.

INCOMPRESSIBLE STRINGS AND RANDOMNESS

Theorem 6.24 shows that a string's minimal description is never much longer than the string itself. Of course for some strings, the minimal description may be much shorter if the information in the string appears sparsely or redundantly. Do some strings lack short descriptions? In other words, is the minimal description of some strings actually as long as the string itself? We show that such strings exist. These strings can't be described any more concisely than simply writing them out explicitly.

DEFINITION 6.28

Let x be a string. Say that x is ***c-compressible*** if

$$K(x) \leq |x| - c.$$

If x is not *c-compressible*, we say that x is ***incompressible by c***.

If x is incompressible by 1, we say that x is ***incompressible***.

In other words, if x has a description that is c bits shorter than its length, x is *c-compressible*. If not, x is incompressible by c . Finally, if x doesn't have any description shorter than itself, x is incompressible. We first show that incompressible strings exist, and then we discuss their interesting properties. In particular, we show that incompressible strings look like strings that are obtained from random coin tosses.

THEOREM 6.29

Incompressible strings of every length exist.

PROOF IDEA The number of strings of length n is greater than the number of descriptions of length less than n . Each description describes at most one string. Therefore, some string of length n is not described by any description of length less than n . That string is incompressible.

PROOF The number of binary strings of length n is 2^n . Each description is a binary string, so the number of descriptions of length less than n is at most the sum of the number of strings of each length up to $n - 1$, or

$$\sum_{0 \leq i \leq n-1} 2^i = 1 + 2 + 4 + 8 + \cdots + 2^{n-1} = 2^n - 1.$$

The number of short descriptions is less than the number of strings of length n . Therefore, at least one string of length n is incompressible.

COROLLARY 6.30

At least $2^n - 2^{n-c+1} + 1$ strings of length n are incompressible by c .

PROOF We extend the proof of Theorem 6.29. Every c -compressible string has a description of length at most $n - c$. No more than $2^{n-c+1} - 1$ such descriptions can occur. Therefore, at most $2^{n-c+1} - 1$ of the 2^n strings of length n may have such descriptions. The remaining strings, numbering at least $2^n - (2^{n-c+1} - 1)$, are incompressible by c .

.....

Incompressible strings have many properties that we would expect to find in randomly chosen strings. For example, we can show that any incompressible string of length n has roughly an equal number of 0s and 1s, and that the length of its longest run of 0s is approximately $\log_2 n$, as we would expect to find in a random string of that length. Proving such statements would take us too far afield into combinatorics and probability, but we will prove a theorem that forms the basis for these statements.

That theorem shows that any computable property that holds for “almost all” strings also holds for all sufficiently long incompressible strings. As we mentioned in Section 0.2, a **property** of strings is simply a function f that maps strings to $\{\text{TRUE}, \text{FALSE}\}$. We say that a property **holds for almost all strings** if the fraction of strings of length n on which it is FALSE approaches 0 as n grows large. A randomly chosen long string is likely to satisfy a computable property that holds for almost all strings. Therefore, random strings and incompressible strings share such properties.

THEOREM 6.31

Let f be a computable property that holds for almost all strings. Then, for any $b > 0$, the property f is FALSE on only finitely many strings that are incompressible by b .

PROOF Let M be the following algorithm.

$M =$ “On input i , a binary integer:

1. Find the i th string s where $f(s) = \text{FALSE}$, in the standard string order.
2. Output string s .”

We can use M to obtain short descriptions of strings that fail to have property f as follows. For any such string x , let i_x be the position or **index** of x on a list of all strings that fail to have property f , in the standard string order (i.e., by length and lexicographically within each length). Then $\langle M, i_x \rangle$ is a description of x . The length of this description is $|i_x| + c$, where c is the length of $\langle M \rangle$. Because few strings fail to have property f , the index of x is small and its description is correspondingly short.

Fix any number $b > 0$. Select n such that at most a $1/2^{b+c+1}$ fraction of strings of length n or less fail to have property f . All sufficiently large n satisfy this condition because f holds for almost all strings. Let x be a string of length n that fails to have property f . We have $2^{n+1} - 1$ strings of length n or less, so

$$i_x \leq \frac{2^{n+1} - 1}{2^{b+c+1}} \leq 2^{n-b-c}.$$

Therefore, $|i_x| \leq n - b - c$, so the length of $\langle M, i_x \rangle$ is at most $(n - b - c) + c = n - b$, which implies that

$$K(x) \leq n - b.$$

Thus every sufficiently long x that fails to have property f is compressible by b . Hence only finitely many strings that fail to have property f are incompressible by b , and the theorem is proved.

At this point, exhibiting some examples of incompressible strings would be appropriate. However, as Problem 6.23 asks you to show, the K measure of complexity is not computable. Furthermore, no algorithm can decide in general whether strings are incompressible, by Problem 6.24. Indeed, by Problem 6.25, no infinite subset of them is Turing-recognizable. So we have no way to obtain long incompressible strings and would have no way to determine whether a string is incompressible even if we had one. The following theorem describes certain strings that are nearly incompressible, although it doesn't provide a way to exhibit them explicitly.

THEOREM 6.32

For some constant b , for every string x , the minimal description $d(x)$ of x is incompressible by b .

PROOF Consider the following TM M :

$M =$ “On input $\langle R, y \rangle$, where R is a TM and y is a string:

1. Run R on y and *reject* if its output is not of the form $\langle S, z \rangle$.
2. Run S on z and halt with its output on the tape.”

Let b be $|\langle M \rangle| + 1$. We show that b satisfies the theorem. Suppose to the contrary that $d(x)$ is b -compressible for some string x . Then

$$|d(d(x))| \leq |d(x)| - b.$$

But then $\langle M \rangle d(d(x))$ is a description of x whose length is at most

$$|\langle M \rangle| + |d(d(x))| \leq (b - 1) + (|d(x)| - b) = |d(x)| - 1.$$

This description of x is shorter than $d(x)$, contradicting the latter's minimality.

EXERCISES

- 6.1 Give an example in the spirit of the recursion theorem of a program in a real programming language (or a reasonable approximation thereof) that prints itself out.
- 6.2 Show that any infinite subset of MIN_{TM} is not Turing-recognizable.
- ^A6.3 Show that if $A \leq_T B$ and $B \leq_T C$, then $A \leq_T C$.
- 6.4 Let $A_{TM}' = \{\langle M, w \rangle \mid M \text{ is an oracle TM and } M^{A_{TM}} \text{ accepts } w\}$. Show that A_{TM}' is undecidable relative to A_{TM} .
- ^A6.5 Is the statement $\exists x \forall y [x+y=y]$ a member of $Th(\mathcal{N}, +)$? Why or why not? What about the statement $\exists x \forall y [x+y=x]$?

PROBLEMS

- 6.6 Describe two different Turing machines, M and N , where M outputs $\langle N \rangle$ and N outputs $\langle M \rangle$, when started on any input.
- 6.7 In the fixed-point version of the recursion theorem (Theorem 6.8), let the transformation t be a function that interchanges the states q_{accept} and q_{reject} in Turing machine descriptions. Give an example of a fixed point for t .
- *6.8 Show that $EQ_{TM} \not\leq_m \overline{EQ_{TM}}$.
- ^A6.9 Use the recursion theorem to give an alternative proof of Rice's theorem in Problem 5.28.
- ^A6.10 Give a model of the sentence

$$\begin{aligned} \phi_{\text{eq}} = & \forall x [R_1(x, x)] \\ & \wedge \forall x, y [R_1(x, y) \leftrightarrow R_1(y, x)] \\ & \wedge \forall x, y, z [(R_1(x, y) \wedge R_1(y, z)) \rightarrow R_1(x, z)]. \end{aligned}$$

- *6.11 Let ϕ_{eq} be defined as in Problem 6.10. Give a model of the sentence

$$\begin{aligned} \phi_{\text{lt}} = & \phi_{\text{eq}} \\ & \wedge \forall x, y [R_1(x, y) \rightarrow \neg R_2(x, y)] \\ & \wedge \forall x, y [\neg R_1(x, y) \rightarrow (R_2(x, y) \oplus R_2(y, x))] \\ & \wedge \forall x, y, z [(R_2(x, y) \wedge R_2(y, z)) \rightarrow R_2(x, z)] \\ & \wedge \forall x \exists y [R_2(x, y)]. \end{aligned}$$

- ^A6.12 Let $(\mathcal{N}, <)$ be the model with universe $\mathcal{N}$ and the “less than” relation. Show that $Th(\mathcal{N}, <)$ is decidable.

- 6.13 For each $m > 1$ let $\mathcal{Z}_m = \{0, 1, 2, \dots, m-1\}$, and let $\mathcal{F}_m = (\mathcal{Z}_m, +, \times)$ be the model whose universe is $\mathcal{Z}_m$ and that has relations corresponding to the $+$ and $\times$ relations computed modulo m . Show that for each m , the theory $\text{Th}(\mathcal{F}_m)$ is decidable.
- 6.14 Show that for any two languages A and B , a language J exists, where $A \leq_T J$ and $B \leq_T J$.
- 6.15 Show that for any language A , a language B exists, where $A \leq_T B$ and $B \not\leq_T A$.
- *6.16 Prove that there exist two languages A and B that are Turing-incomparable—that is, where $A \not\leq_T B$ and $B \not\leq_T A$.
- *6.17 Let A and B be two disjoint languages. Say that language C *separates* A and B if $A \subseteq C$ and $B \subseteq \overline{C}$. Describe two disjoint Turing-recognizable languages that aren't separable by any decidable language.
- 6.18 Show that $\overline{EQ_{TM}}$ is recognizable by a Turing machine with an oracle for A_{TM} .
- 6.19 In Corollary 4.18, we showed that the set of all languages is uncountable. Use this result to prove that languages exist that are not recognizable by an oracle Turing machine with an oracle for A_{TM} .
- 6.20 Recall the Post Correspondence Problem that we defined in Section 5.2 and its associated language PCP . Show that PCP is decidable relative to A_{TM} .
- 6.21 Show how to compute the descriptive complexity of strings $K(x)$ with an oracle for A_{TM} .
- 6.22 Use the result of Problem 6.21 to give a function f that is computable with an oracle for A_{TM} , where for each n , $f(n)$ is an incompressible string of length n .
- 6.23 Show that the function $K(x)$ is not a computable function.
- 6.24 Show that the set of incompressible strings is undecidable.
- 6.25 Show that the set of incompressible strings contains no infinite subset that is Turing-recognizable.
- *6.26 Show that for any c , some strings x and y exist, where $K(xy) > K(x) + K(y) + c$.
- 6.27 Let $S = \{\langle M \rangle \mid M \text{ is a TM and } L(M) = \{\langle M \rangle\}\}$. Show that neither S nor $\overline{S}$ is Turing-recognizable.
- 6.28 Let $R \subseteq \mathcal{N}^k$ be a k -ary relation. Say that R is **definable** in $\text{Th}(\mathcal{N}, +)$ if we can give a formula ϕ with k free variables $x_1, \dots, x_k$ such that for all $a_1, \dots, a_k \in \mathcal{N}$, $\phi(a_1, \dots, a_k)$ is true exactly when $a_1, \dots, a_k \in R$. Show that each of the following relations is definable in $\text{Th}(\mathcal{N}, +)$.
- $R_0 = \{0\}$
 - $R_1 = \{1\}$
 - $R_+ = \{(a, a) \mid a \in \mathcal{N}\}$
 - $R_< = \{(a, b) \mid a, b \in \mathcal{N} \text{ and } a < b\}$



SELECTED SOLUTIONS

- 6.3 Say that M_1^B decides A and M_2^C decides B . Use an oracle TM M_3 , where M_3^C decides A . Machine M_3 simulates M_1 . Every time M_1 queries its oracle about some string x , machine M_3 tests whether $x \in B$ and provides the answer to M_1 . Because machine M_3 doesn't have an oracle for B and cannot perform that test directly, it simulates M_2 on input x to obtain that information. Machine M_3 can obtain the answer to M_2 's queries directly because these two machines use the same oracle, C .
- 6.5 The statement $\exists x \forall y [x+y=y]$ is a member of $\text{Th}(\mathcal{N}, +)$ because that statement is true for the standard interpretation of $+$ over the universe $\mathcal{N}$. Recall that we use $\mathcal{N} = \{0, 1, 2, \dots\}$ in this chapter and so we may use $x = 0$. The statement $\exists x \forall y [x+y=x]$ is not a member of $\text{Th}(\mathcal{N}, +)$ because that statement isn't true in this model. For any value of x , setting $y = 1$ causes $x+y=x$ to fail.
- 6.9 Assume for the sake of contradiction that some TM X decides a property P , and P satisfies the conditions of Rice's theorem. One of these conditions says that TMs A and B exist where $\langle A \rangle \in P$ and $\langle B \rangle \notin P$. Use A and B to construct TM R :

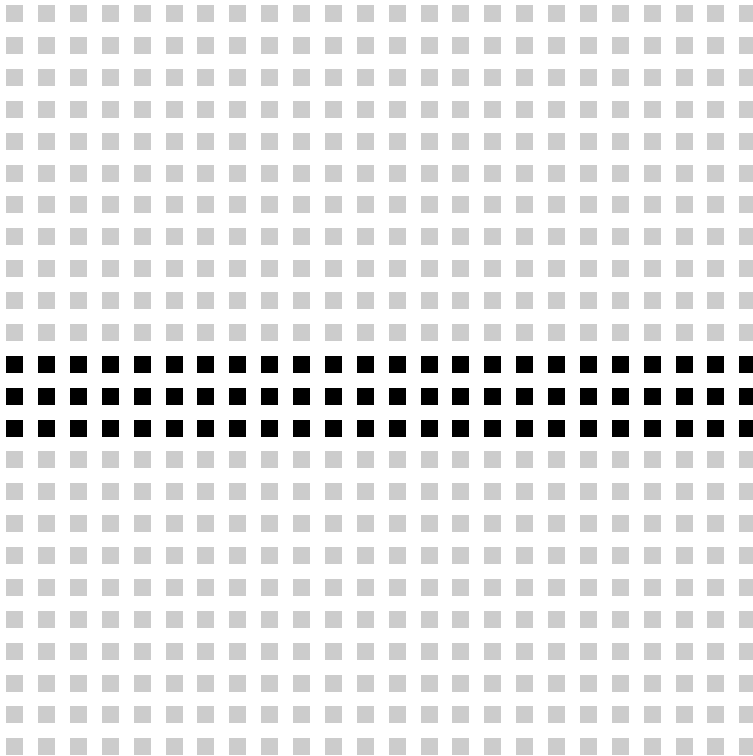
$R =$ "On input w :

1. Obtain own description $\langle R \rangle$ using the recursion theorem.
2. Run X on $\langle R \rangle$.
3. If X accepts $\langle R \rangle$, simulate B on w .
If X rejects $\langle R \rangle$, simulate A on w ."

If $\langle R \rangle \in P$, then X accepts $\langle R \rangle$ and $L(R) = L(B)$. But $\langle B \rangle \notin P$, contradicting $\langle R \rangle \in P$, because P agrees on TMs that have the same language. We arrive at a similar contradiction if $\langle R \rangle \notin P$. Therefore, our original assumption is false. Every property satisfying the conditions of Rice's theorem is undecidable.

- 6.10 The statement ϕ_{eq} gives the three conditions of an equivalence relation. A model (A, R_1) , where A is any universe and R_1 is any equivalence relation over A , is a model of ϕ_{eq} . For example, let A be the integers $\mathcal{Z}$ and let $R_1 = \{(i, i) \mid i \in \mathcal{Z}\}$.
- 6.12 Reduce $\text{Th}(\mathcal{N}, <)$ to $\text{Th}(\mathcal{N}, +)$, which we've already shown to be decidable. Show how to convert a sentence ϕ_1 over the language of $(\mathcal{N}, <)$ to a sentence ϕ_2 over the language of $(\mathcal{N}, +)$ while preserving truth or falsity in the respective models. Replace every occurrence of $i < j$ in ϕ_1 with the formula $\exists k [(i+k=j) \wedge (k+k \neq k)]$ in ϕ_2 , where k is a different new variable each time.
- Sentence ϕ_2 is equivalent to ϕ_1 because " i is less than j " means that we can add a nonzero value to i and obtain j . Putting ϕ_2 into prenex-normal form, as required by the algorithm for deciding $\text{Th}(\mathcal{N}, +)$, requires a bit of additional work. The new existential quantifiers are brought to the front of the sentence. To do so, these quantifiers must pass through Boolean operations that appear in the sentence. Quantifiers can be brought through the operations of $\wedge$ and $\vee$ without change. Passing through $\neg$ changes $\exists$ to $\forall$ and vice versa. Thus, $\neg \exists k \psi$ becomes the equivalent expression $\forall k \neg \psi$, and $\neg \forall k \psi$ becomes $\exists k \neg \psi$.
- 6.28 (a) R_0 is definable in $\text{Th}(\mathcal{N}, +)$ by $\phi_0(x) = \forall y [x + y = y]$.
(c) $R_ =$ is definable in $\text{Th}(\mathcal{N}, +)$ by $\phi_=(u, v) = \forall x [\phi_0(x) \rightarrow x + u = v]$.

PART THREE



C O M P L E X I T Y T H E O R Y

